

Reviewer Pack — Minimal Rank of Noncommutative Crossed Products Bounds BP Re...

Entry d8251b5e8cc9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 20:31:47 UTC

Minimal Rank of Noncommutative Crossed Products Bounds BP Read-Twice Width

Entry ID: d8251b5e8cc9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 20:31:47 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Noncommutative Crossed Products)

Field B (complexity object): Branching Program: Read-twice Complexity

Statement:

{'part1': 'Let P be a read-twice branching program with n inputs, and let $M = C[0,1]^n \otimes F^2$ be the crossed product algebra of the unit circle algebra $C[0,1]^n$ by the two-sided ideal generated by the unitary element $u(\alpha) = \exp(i\alpha)$ for α in $[0, \pi]$.', 'part2': 'The minimal rank of the noncommutative crossed product algebra M is equal to $O(\log \text{width}(P))$ but grows at least linearly with n when P is a trivial read-twice BP.', 'part3': 'For a trivial read-twice BP of size 1, the rank of M is $O(1)$.')}

Rationale (proposer's reasoning):

{'part1': 'Noncommutative crossed products have been used to encode complex systems in mathematics and physics. Their minimal rank may capture the complexity of computation in read-twice BPs.', 'part2': 'If this conjecture is true, it would provide a new way to understand the complexity of read-twice BP computations that does not rely on classical algebraic tools.', 'part3': 'This approach has not been explored in complexity theory and could potentially lead to novel insights.'}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f9a7eec3ac6320da

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for a trivial read-twice BP with size n , the minimal rank of M grows at least linearly with n and is within $O(\log \text{width}(P))$ of the width of P .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for j in range(cols):
        if all(matrix[i][j] == 0 for i in range(rank)):
            continue
        pivot_row = rank
        for i in range(pivot_row + 1, rows):
            if matrix[i][j] != 0:
                matrix[pivot_row], matrix[i] = matrix[i], matrix[pivot_row]
                break
        factor = Fraction(matrix[pivot_row][j])
        for k in range(cols):
            matrix[pivot_row][k] /= factor
        for i in range(rows):
            if i != pivot_row and matrix[i][j] != 0:
                factor = Fraction(matrix[i][j])
                for k in range(cols):
                    matrix[i][k] -= factor * matrix[pivot_row][k]
        rank += 1
    return rank
```

```

def generate_read_twice_bp(n, seed):
    random.seed(seed)
    bp = []
    for _ in range(2 ** n):
        bp.append(random.choice([0, 1]))
    return bp

def construct_crossed_product_algebra(bp, n):
    F = [[Fraction(i == j, 1) for j in range(n)] for i in range(n)]
    u_alpha = [[Fraction(math.cos(alpha), 1), Fraction(-math.sin(alpha), 1)], [Fraction(math.sin(alpha), 1), Fraction(-math.cos(alpha), 1)]]
    M = []
    for row in F:
        new_row = []
        for col in row:
            new_col = []
            for i in range(n):
                temp = []
                for j in range(n):
                    temp.append(col * u_alpha[i][j])
                new_col.extend(temp)
            new_row.extend(new_col)
        M.extend(new_row)
    return M

def run_trial(seed: int) -> dict:
    n = random.choice([5, 10, 15, 20, 30, 40])
    bp = generate_read_twice_bp(n, seed)
    M = construct_crossed_product_algebra(bp, n)
    rank_M = matrix_rank(M)
    metric_value = rank_M
    instances_tested = 1
    conjecture_holds = rank_M >= n and rank_M <= math.log(2 ** n, 2)
    counterexample = "" if conjecture_holds else f"n={n}, rank_M={rank_M}"
    return {
        "metric_name": "Minimal Rank of Crossed Product",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) # Default to 1
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"instances_tested\": {trial_result['instances_tested']}, \"conjecture_holds\": {trial_result['conjecture_holds']}, \"counterexample\": \"{trial_result['counterexample']}\"}}")
        results.append(trial_result)

    mean_metric = sum(result["metric_value"] for result in results) / len(results)
    std_metric = math.sqrt(sum((result["metric_value"] - mean_metric) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con]
    print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['instances_tested']}, rank_M={res

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify whether the conjecture's conditions are met. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12648
2	preregistration	ollama_remote	glm4:latest	0	0	5709
3	novelty	ollama_remote	glm4:latest	0	0	4821
4	novelty	ollama_remote	glm4:latest	0	0	5048
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15742
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9854
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11743
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12258
9	judge	ollama_remote	glm4:latest	0	0	14370

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92193 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/d8251b5e8cc9.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d8251b5e8cc9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d8251b5e8cc9.tar.gz` (if generated)